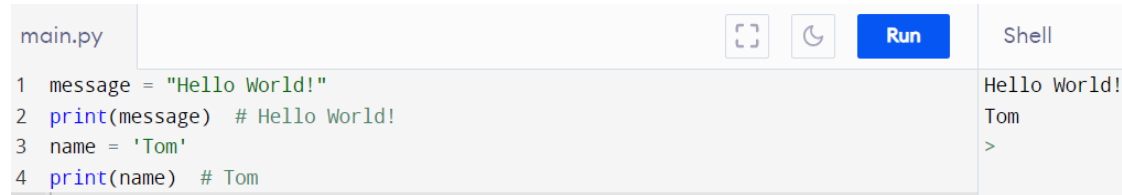


Тема 8. Низ. Работа с низ. Основни методи. Форматиране

Низ. Работа с низ

Низът е поредица от символи на Unicode, затворени в кавички. Освен това Python позволява използването, както на единични, така и на двойни кавички за дефиниране на низове:

```
message = "Hello World!"  
print(message) # Hello World!  
name = 'Tom'  
print(name) # Tom
```



```
main.py  
1 message = "Hello World!"  
2 print(message) # Hello World!  
3 name = 'Tom'  
4 print(name) # Tom
```

Shell

```
Hello World!  
Tom  
>
```

Ако редът е дълъг, може да го разделите на части и да ги поставите на различни редове код. В този случай целият низ е затворен в скоби, а отделните му части са в кавички:

```
text = ("Laude omnes gentes laude "  
       "Magnificat in secula ")  
print(text)
```

Ако трябва да се дефинира многоредов текст, тогава такъв текст е затворен в тройни двойни или единични кавички:

```
"""  
Това е коментар  
"""  
  
text = """Laude omnes gentes laude  
Magnificat in secula  
Et anima mea laude  
Magnificat in secula  
"""  
  
print(text)
```



```
main.py  
1 """  
2 Това е коментар  
3 """  
4 text = """Laude omnes gentes laude  
5 Magnificat in secula  
6 Et anima mea laude  
7 Magnificat in secula  
8 """
```

Shell

```
Laude omnes gentes laude  
Magnificat in secula  
Et anima mea laude  
Magnificat in secula  
  
> |
```

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

Когато се използват тройни единични кавички, не трябва да се бъркат с коментари: ако текстът в тройни единични кавички е присвоен на променлива, тогава това е низ, а не коментар.

Escape последователности в низ

Низът може да съдържа редица специални символи - ескаре-последователности. Някои от тях:

- `\:` позволява да се добави наклонена черта вътре в низа
- `\'`: позволява да се добави единична кавичка вътре в низа
- `\"`: позволява да се добавят двойни кавички вътре в низа
- `\n`: Преминава към нов ред
- `\t`: добавя раздел (4 отстъпа)

Нека се разгледат някои последователности:

```
text = "Message:\n\"Hello World\\""
print(text)
```

Конзолен изход на програмата:

```
Message:
```

```
"Hello World"
```

Подобни последователности могат да помогнат при поставяне на цитат в низ, при направа на таблица, за преход на нов ред, но могат и да пречат. **Пример:**

```
path = "C:\python\name.txt"
```

```
print(path)
```

Променливата **path** съдържа път до файла, въпреки че, в низа има символи `"\n"`, които ще се интерпретират, като escape последователност, което ще доведе до следния конзолен изход:

```
c:\python
```

```
ame.txt
```

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

За да се избегне подобна ситуация, символът **r** се поставя преди низа.

```
path = r"C:\python\name.txt"
print(path)
```

Вмъкване на стойности в низ

Python позволява да се вграждат стойностите на други променливи в низ. За да се направи това, вътре в низа, променливите се поставят в къдрави скоби {}, а символът f се поставя преди целия низ :

main.py	  	Shell
<pre>1 userName = "Tom" 2 userAge = 37 3 user = f"name: {userName} age: {userAge}" 4 print(user) # name: Tom age: 37</pre>		<pre>name: Tom age: 37 ></pre>

В този случай, стойността на променливата `userName` ще бъде вмъкната на място `{userName}`, а стойността на променливата `userAge` ще бъде вмъкната на място `{userAge}`.

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

Референтни знаци в низ

Може да се получи достъп до отделните знаци на низ чрез индекс в квадратни скоби. Индексирането започва от нула, така че първият символ на низа ще има индекс 0. И ако се направи опит да се осъществи достъп до индекс, който не е в низа, ще се получи изключение **IndexError**. Например, в горния случай низът е дълъг 11 знака, така че неговите знаци ще имат индекси от 0 до 10. За достъп до знаци, започващи от края на низ, могат да се използват отрицателни индекси. И така, индекс -1 ще представлява последния знак, а -2 ще представлява предпоследния знак и т.н.:

```
string = "hello world"
```

```
c0 = string[0] # h
```

```
print(c0)
```

```
c6 = string[6] # w
```

```
print(c6)
```

```
c11 = string[11] # грешка IndexError: string index out of range
```

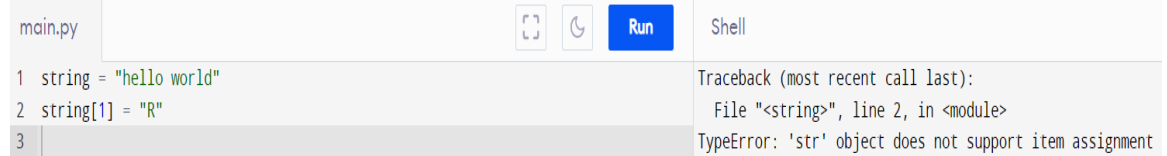
```
print(c11)
```

main.py	Run	Shell
<pre>1 string = "hello world" 2 c0 = string[0] # h 3 print(c0) 4 c6 = string[6] # w 5 print(c6) 6 c11 = string[11] # грешка IndexError: string index out of range 7 print(c11)</pre>		<pre>h w Traceback (most recent call last): File "<string>", line 6, in <module> IndexError: string index out of range > </pre>

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

Когато се работи със знаци, трябва да се има предвид, че **низът е неизменяем тип**, така че ако се направи опит да се промени всеки отделен знак от низ, ще се получи грешка, както в следния случай:

```
string = "hello world"
string[1] = "R"
```



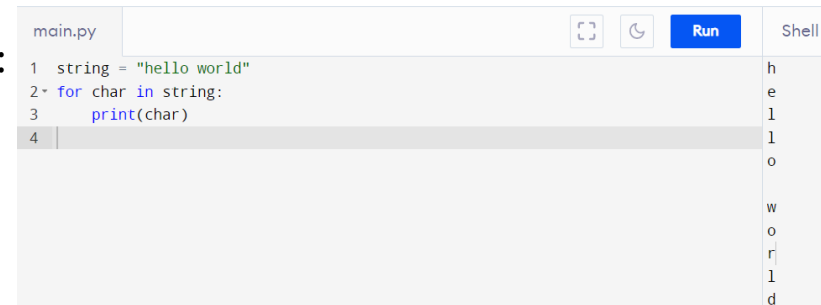
```
main.py
1 string = "hello world"
2 string[1] = "R"
3
Shell
Traceback (most recent call last):
  File "<string>", line 2, in <module>
TypeError: 'str' object does not support item assignment
```

Може да се нулира напълно стойността на низ само като му се присвои различна стойност.

Итерация на линия

Може да се премине през всички знаци в низ с цикъл for :

```
string = "hello world"
for char in string:
    print(char)
```



```
main.py
1 string = "hello world"
2 for char in string:
3     print(char)
4
Shell
h
e
l
l
o
w
o
r
l
d
```

Получаване на подниз

Ако е необходимо, може да се получат не само отделните символи от низ, но и подниз. Затова се използва следният синтаксис:

- **string[:end]:** извлича поредица от знаци от индекс 0 до края на индекса (без да се включва)
- **string[start:end]:** извлича поредица от знаци от началото на индекса до края на индекса (без да се включва)
- **string[start:end:step]:** извличане на последователността от знаци от началото на индекса до края на индекса (без да се включва) чрез стъпка

Използват се всички опции за получаване на подниз:

```
string = "hello world"
```

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

от 0 до 5 индекс

```
sub_string1 = string[:5]
```

```
print(sub_string1)    # hello
```

от 2 до 5 индекс

```
sub_string2 = string[2:5]
```

```
print(sub_string2)    # llo
```

от 2 до 9 индекс през един знак

```
sub_string3 = string[2:9:2]
```

```
print(sub_string3)    # lowr
```

```
main.py  [ ] [ ] Run Shell
1 string = "hello world"
2 # от 0 до 5 индекс
3 sub_string1 = string[:5]
4 print(sub_string1)    # hello
5
6 # от 2 до 5 индекс
7 sub_string2 = string[2:5]
8 print(sub_string2)    # llo
9
10 # от 2 до 9 индекс през един знак
11 sub_string3 = string[2:9:2]
12 print(sub_string3)    # lowr
```

hello
llo
lowr
>

Конкатенация на низове

Една от най-често срещаните операции върху низове е тяхното обединяване или конкатенация. Операцията по добавяне се използва за конкатениране на низове:

```
name = "Tom"
```

```
surname = "Smith"
```

```
fullname = name + " " + surname
```

```
print(fullname) # Tom Smith
```

```
main.py  [ ] [ ] Run Shell
1 name = "Tom"
2 surname = "Smith"
3 fullname = name + " " + surname
4 print(fullname) # Tom Smith
```

Tom Smith
>

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

Свързването на два низа е лесно, но какво ще стане, ако трябва да се добави низ и число? В този случай трябва да се **прехвърли числото към низ с помощта на функцията str():**

```
name = "Tom"
age = 33
info = "Name: " + name + " Age: " + str(age)
print(info) # Name: Tom Age: 33
```

main.py	  	Shell
<pre>1 name = "Tom" 2 age = 33 3 info = "Name: " + name + " Age: " + str(age) 4 print(info) # Name: Tom Age: 33</pre>		<pre>Name: Tom Age: 33 > </pre>

Повторение на линия

За да се повтори низ определен брой пъти, се използва операцията умножение:

```
print("a" * 3) # aaa
print("he" * 4) # hehehehe
```

main.py	  	Shell
<pre>1 print("a" * 3) # aaa 2 print("he" * 4) # hehehehe</pre>		<pre>aaa hehehehe</pre>

Сравнение на низове

Специално трябва да се спомене сравненията на низове. При сравняване на низове се вземат предвид знаците и техният регистър. И така, цифровият знак е условно по-малък от всеки азбучен знак. Главните букви на азбуката обикновено са по-малки от малките букви. Например:

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

```
str1 = "1a"  
str2 = "aa"  
str3 = "Aa"
```

```
print(str1 > str2) #  
print(str2 > str3) #
```

main.py		Run	Shell
1	str1 = "1a"		False
2	str2 = "aa"		True
3	str3 = "Aa"		>
4	print(str1 > str2) # False, тъй като първият знак в str1 е цифра		
5	print(str2 > str3) # True, тъй като първият знак в str2 е с малки букви		

Следователно низът "1a" е условно по-малък от низа "aa". Първият знак се сравнява първи. Ако водещите символи на двата низа представляват цифри, тогава по-малката цифра се счита за по-малка, например "1a" е по-малко от "2a". Ако първоначалните знаци представляват азбучни знаци в същия случай, тогава потърсете по азбучен ред. Така че "aa" е по-малко от "ba", а "ba" е по-малко от "ca". Ако първите знаци са еднакви, вторите знаци, ако има такива, се вземат предвид. Чувствителността на малки и големи букви не винаги е желателна, тъй като всъщност работата е с едни и същи низове. В този случай, преди да се направи сравнението, може да се прехвърлят двата низа към един от регистрите.

Функцията `lower()` преобразува низ в малки букви, а функцията `upper()` го преобразува в главни букви.

```
str1 = "Tom"  
str2 = "tom"
```

```
print(str1 == str2) # False - струните не са равни
```

main.py		Run	Shell
1	str1 = "Tom"		False
2	str2 = "tom"		>
3	print(str1 == str2) # False - струните не са равни		

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

```
print(str1.lower() == str2.lower()) # True
```

функции `ord` и `len`

Тъй като низът съдържа Unicode символи, можем да се използва функцията `ord()`, за да се получи числовата стойност за Unicode символ:

```
print(ord("A")) # 65
```

Може да се използва функцията `len()`, за да се получи дължината на низа:

```
string = "hello world"
```

```
length = len(string)
```

```
print(length) # 11
```

main.py	Run	Shell
<pre>1 str1 = "Tom" 2 str2 = "tom" 3 print(str1.lower() == str2.lower()) 4 5 print(ord("A")) # 65 6 7 string = "hello world" 8 length = len(string) 9 print(length) # 11</pre>		<pre>True 65 11 > </pre>

Търсене в низ

С помощта на израз `term in string` може да се намери `term`-а на подниз в друг низ. Ако поднизът бъде намерен, тогава изразът ще върне стойността `True`, в противен случай стойността ще бъде върната `False`:

```
string = "hello world"
```

```
exist = "hello" in string
```

```
print(exist) # True
```

```
exist = "sword" in string
```

```
print(exist) # False
```

main.py	Run	Shell
<pre>1 string = "hello world" 2 exist = "hello" in string 3 print(exist) # True 4 5 exist = "sword" in string 6 print(exist) # False</pre>		<pre>True False > </pre>

Основни низови методи

Основните низови методи, които могат да се приложат са:

- **isalpha()**: връща True, ако низът се състои само от азбучни знаци
- **islower()**: връща True, ако низът се състои само от малки букви
- **isupper()**: връща True, ако всички знаци в низа са главни
- **isdigit()**: връща True, ако всички знаци в низ са цифри
- **isnumeric()**: връща True, ако низът е число
- **startswith(str)**: връща True, ако низът започва с подниз str
- **endswith(str)**: връща True, ако низът завършва с подниз str
- **low()**: преобразува низ в малки букви
- **upper()**: преобразува низ в главни букви
- **title()**: началните знаци на всички думи в низа се преобразуват в главни букви

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

- **title()**: началните знаци на всички думи в низа се преобразуват в главни букви
- **capitalize()**: главна буква само на първата дума от низа
- **lstrip()**: премахва водещите интервали от низ
- **rstrip()**: премахва крайните интервали от низ
- **strip()**: премахва началните и крайните интервали от низ
- **ljust(width)**: ако дължината на низа е по-малка от параметъра ширина, отдясно на низа се добавят интервали, за да допълнят стойността на ширината, а самият низ се подравнява вляво
- **rjust(width)**: ако дължината на низа е по-малка от параметъра ширина, отляво на низа се добавят интервали, за да допълнят стойността на ширината, а самият низ е подравнен вдясно
- **center(width)**: ако дължината на низа е по-малка от параметъра ширина, интервали се добавят равномерно отляво и отдясно на низа, за да допълнят стойността на ширината, а самият низ е центриран
- **find(str[, start [, end]])**: връща индекса на подниз в низ. Ако поднизът не бъде намерен, се връща числото -1
- **replace(old, new[, num])**: замества един подниз в низ с друг
- **split([delimiter[, num]])**: разделя низ на поднизове в зависимост от разделителя
- **join(strs)**: конкатенира низове в един низ, чрез вмъкване на специфичен разделител между тях.

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

Например, ако числото вече е въведено от клавиатурата, тогава преди да се преобразува въведеното в число, може да се провери дали числото е валидно с помощта на метода `isnumeric()` и така ще се изпълни операцията за преобразуване:

```
string = input("Въведете число: ")
```

```
if string.isnumeric():
```

```
    number = int(string)
```

```
    print(number)
```

Проверка дали низ започва или завършва с конкретен подниз:

```
file_name = "hello.py"
```

```
starts_with_hello = file_name.startswith("hello")
```

```
ends_with_exe = file_name.endswith("exe")
```

```
print(starts_with_hello)
```

```
print(ends_with_exe)
```

Премахване на интервали в началото и края на низ:

```
string = "  hello world! "
```

```
string = string.strip()
```

```
print(string)          # hello world!
```

Допълване на низ с интервали и подравняване:

```
print("iPhone 7:", "52000".rjust(10))
```

```
print("Huawei P10:", "36000".rjust(10))
```

Изход на конзолата:

iPhone 7: 52000

Huawei P10: 36000

```
main.py  Run Shell
1 string = input("Въведете число: ")
2 if string.isnumeric():
3     number = int(string)
4     print(number)
```

Въведете число: 53
53
> |

```
main.py  Run Shell
1 file_name = "hello.py"
2 starts_with_hello = file_name.startswith("hello") # True
3 ends_with_exe = file_name.endswith("exe")         # False
4 print(starts_with_hello)
5 print(ends_with_exe)
```

True
False
> |

```
main.py  Run Shell
1 string = "  hello world! "
2 string = string.strip()
3 print(string)          # hello world!
```

hello world!
> |

```
main.py  Run Shell
1 print("iPhone 7:", "52000".rjust(10))
2 print("Huawei P10:", "36000".rjust(10))
```

iPhone 7: 52000
Huawei P10: 36000

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

Търсене в низ

За да се намери подниз в низ, Python използва метода `find()`, който връща индекса на първото появяване на подниз в низ и има три форми:

- **`find(str)`**: подниз `str` се търси от началото на низа до края му
- **`find(str, start)`**: началният параметър определя началния индекс, от който да се търси
- **`find(str, start, end)`**: параметърът `end` определя крайния индекс, към който ще отиде търсенето

Ако поднизът не бъде намерен, методът връща `-1`:

```
welcome = "Hello world! Goodbye world!"
```

```
index = welcome.find("wor")
```

```
print(index)    # 6
```

```
# търсене от индекс 10
```

```
index = welcome.find("wor",10)
```

```
print(index)    # 21
```

```
# търсене от 10-ти до 15-ти индекс
```

```
index = welcome.find("wor",10,15)
```

```
print(index)    # -1
```



```
main.py
1 welcome = "Hello world! Goodbye world!"
2 index = welcome.find("wor")
3 print(index)    # 6
4 # търсене от индекс 10
5 index = welcome.find("wor",10)
6 print(index)    # 21
7 # търсене от 10-ти до 15-ти индекс
8 index = welcome.find("wor",10,15)
9 print(index)    # -1
```

	Shell
1	6
2	21
3	-1
4	>

Замяна в низ

За да замените един подниз в низ с друг, използвайте метода `replace()` :

- **`replace(old, new)`**: замества подниза `old` с нов
- **`replace(old, new, num)`**: параметърът `num` указва колко пъти поява на подниза `old` да се замени с `new`

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

```
phone = "+1-234-567-89-10"
# замяна на тирета с интервали
edited_phone = phone.replace("-", " ")
print(edited_phone)    # +1 234 567 89 10
# премахване на тире
edited_phone = phone.replace("-", "")
print(edited_phone)    # +12345678910
# заместване само на първото тире
edited_phone = phone.replace("-", "", 1)
print(edited_phone)    # +1234-567-89-10
```

main.py	Run	Shell
<pre>1 phone = "+1-234-567-89-10" 2 # замяна на тирета с интервали 3 edited_phone = phone.replace("-", " ") 4 print(edited_phone) # +1 234 567 89 10 5 # премахване на тире 6 edited_phone = phone.replace("-", "") 7 print(edited_phone) # +12345678910 8 # заместване само на първото тире 9 edited_phone = phone.replace("-", "", 1) 10 print(edited_phone) # +1234-567-89-10</pre>		<pre>+1 234 567 89 10 +12345678910 +1234-567-89-10 ></pre>

Разделяне на поднизове

Методът `split()` разделя низ на списък с поднизове, в зависимост от разделителя. Разделителят може да бъде произволен знак или последователност от знаци. Този метод има следните форми:

- **`split()`**: пространството се използва като разделител
- **`split(delimiter)`**: разделител се използва като разделител
- **`split(delimiter, num)`**: Параметърът `num` указва колко пъти появата на разделител се използват за разделяне. Останалата част от низа се добавя към списъка, без да се разделя на поднизове.

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

```
text = "Честита Коледа, и Весела Нова 2023 година!"
splitted_text = text.split()
print(splitted_text)      # разделяне на думи
print(splitted_text[5])   # извеждане на поредна дума в изрази
splitted_text = text.split(",")
print(splitted_text)      # извеждане на поредна дума в изрази
print(splitted_text[0])   # извеждане на изрази до запетаята
splitted_text = text.split(" ", 3) # извеждане на думите с разделител празен символ
print(splitted_text)
print(splitted_text[0]) # извеждае на първата дума в низа
```

main.py	Shell
<pre>1 text = "Честита Коледа, и Весела Нова 2023 година!" 2 splitted_text = text.split() 3 print(splitted_text) # разделяне на думи 4 5 print(splitted_text[5]) # извеждане на поредна дума в изрази 6 7 splitted_text = text.split(",") 8 print(splitted_text) # извеждане на поредна дума в изрази 9 10 print(splitted_text[0]) # извеждане на изрази до запетаята 11 12 13 splitted_text = text.split(" ", 3) 14 # извеждане на думите с разделител празен символ 15 16 print(splitted_text) 17 print(splitted_text[0]) 18 # извеждае на първата дума в низа</pre>	<pre>['Честита', 'Коледа,', 'и', 'Весела', 'Нова', '2023', 'година!'] 2023 ['Честита Коледа', ' и Весела Нова 2023 година!'] Честита Коледа ['Честита', 'Коледа,', ' ', 'и Весела Нова 2023 година!'] Честита > </pre>

Съединяване на низове

До тук бяха разгледани най-простите операции върху низове, като бе посочено как да свържем низове с помощта на операцията **text.split()**. Друга възможност за свързване на низове е **методът join()**: той обединява списък от низове. Освен това текущият ред, на който се извиква този метод, се използва като разделител:

```
words = ["Let", "me", "speak", "from", "my", "heart", "in", "English"]
```

```
# разделител - интервал
```

```
sentence = " ".join(words)
```

```
print(sentence) # Let me speak from my heart in English
```

```
# разделител – вертикална черта
```

```
sentence = " | ".join(words)
```

```
print(sentence)
```

main.py	  	Shell
<pre>1 words = ["Let", "me", "speak", "from", "my", "heart", "in", "English"] 2 # разделител - интервал 3 sentence = " ".join(words) 4 print(sentence) # Let me speak from my heart in English 5 6 # разделител - вертикална черта 7 sentence = " ".join(words) 8 print(sentence) # Let me speak from my heart in English</pre>		<pre>Let me speak from my heart in English Let me speak from my heart in English > </pre>

Вместо списък, може да се подаде обикновен низ към метода на присъединяване, след което разделителят ще бъде вмъкнат между знаците на този низ:

```
word = "hello"
```

```
joined_word = "|".join(word)
```

```
print(joined_word) # h|e|l|l|o
```

main.py	  	Shell
<pre>1 word = "hello" 2 joined_word = " ".join(word) 3 print(joined_word) # h e l l o</pre>		<pre>h e l l o > </pre>

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

Форматиране

Вмъкване на някои стойности в низ, може да стане, като се постави префикс на низа с `f` :

```
first_name="Tom"
text = f"Hello, {first_name}."
print(text)    # Hello, Tom.
name="Bob"
age=23
info = f"Name: {name}\t Age: {age}"
print(info)    # Name: Bob Age: 23
```

main.py	  	Shell
<pre>1 first_name="Tom" 2 text = f"Hello, {first_name}." 3 print(text) # Hello, Tom. 4 name="Bob" 5 age=23 6 info = f"Name: {name}\t Age: {age}" 7 print(info) # Name: Bob Age: 23</pre>		<pre>Hello, Tom. Name: Bob Age: 23 > </pre>

В Python има и друг алтернативен начин, който предоставя метода `format()`. Този метод позволява да се форматира низ, като се вмъкват определени стойности в него на мястото на заместители. За вмъкване в низ се използват специални параметри, които са рамкирани с къдрави скоби (`{}`).

Именувани параметри

Във форматирания низ може да се дефинират параметри, в метода може да се използва `format()` за да се предадат стойности за тези параметри:

```
text = "Hello, {first_name}.".format(first_name="Tom")
print(text)    # Hello, Tom.
info = "Name: {name}\t Age: {age}".format(name="Bob", age=23)
print(info)    # Name: Bob Age: 23
```

main.py	  	Shell
<pre>1 text = "Hello, {first_name}.".format(first_name="Tom") 2 print(text) # Hello, Tom. 3 info = "Name: {name}\t Age: {age}".format(name="Bob", age=23) 4 print(info) # Name: Bob Age: 23</pre>		<pre>Hello, Tom. Name: Bob Age: 23 > </pre>

Освен това в метода `format` аргументите се дефинират със същото име като параметрите в низа. Така че, ако параметърът се извика `first_name`, както в първия случай, тогава аргументът, на който е присвоена стойността, също се извиква `first_name`.

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

Параметри по позиция

Можем също така **последователно** да се предаде набор от аргументи към метода `format` и да се вмъкнат тези аргументи в самия низ за форматиране, като се посочи техният брой в къдрави скоби (номерирането започва от нула):

```
info = "Name: {0}\t Age: {1}".format("Bob", 23)
print(info)    # Name: Bob Age: 23
```

В този случай аргументите могат да бъдат вмъкнати в низа няколко пъти:

```
text = "Hello, {0} {0} {0}.".format("Tom")
```



```
main.py  Run  Shell
1 info = "Name: {0}\t Age: {1}".format("Bob", 23)
2 print(info)    # Name: Bob Age: 23
3
4 text = "Hello, {0} {0} {0}.".format("Tom")
5 print(text)
Name: Bob    Age: 23
Hello, Tom Tom Tom.
> |
```

Замени

Друг начин за предаване на форматирувани стойности към низ е използването на замествания или специални заместители, на мястото на които се вмъкват конкретни стойности. За форматиране може да се използват следните заместители:

- **s:** За вмъкване на редове
- **d:** За вмъкване на цели числа
- **f:** За вмъкване на дробни числа. За този тип също така е възможно да се определи броят на десетичните знаци чрез точка
- **%:** Умножава стойността по 100 и добавя знак за процент
- **e:** Отпечатва число в научна нотация

Общият синтаксис за заместител е както следва:

{:плейсхолдер}

Могат да се добавят допълнителни параметри в зависимост от заместителя. Например, за да се форматираат числа с плаваща стойност, може да се използват следните опции

{:[количество символи][запетая][брой цифри в дробната част] плейсхолдер}

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

При извикване на метода `format` стойностите се предават към него като аргументи, които се вмъкват на мястото на заместителите:

```
welcome = "Hello {s}"
name = "Tom"
formatted_welcome = welcome.format(name)
print(formatted_welcome)      # Hello Tom
```



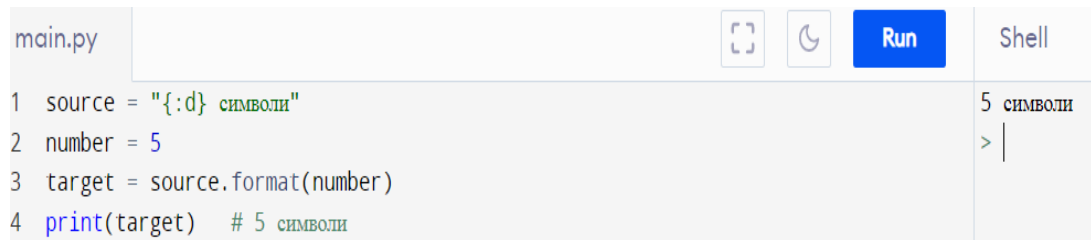
```
main.py
1 welcome = "Hello {s}"
2 name = "Tom"
3 formatted_welcome = welcome.format(name)
4 print(formatted_welcome)      # Hello Tom
```

Shell

```
Hello Tom
>
```

В резултат на това методът `format()` връща нов форматиран низ. Форматиране на цели числа:

```
source = "{:d} символи"
number = 5
target = source.format(number)
print(target)  # 5 символи
```



```
main.py
1 source = "{:d} символи"
2 number = 5
3 target = source.format(number)
4 print(target)  # 5 символи
```

Shell

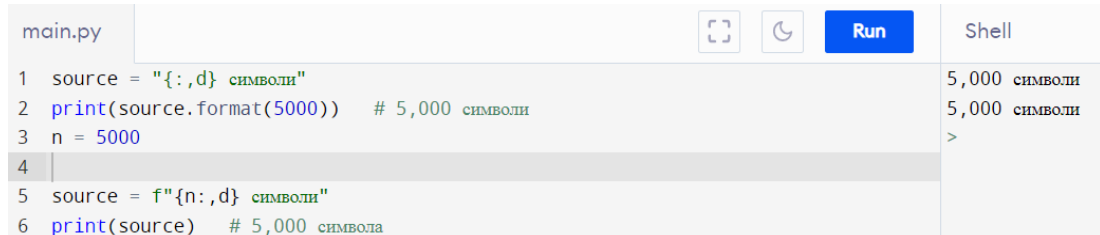
```
5 символи
> |
```

Ако числото, което трябва да се форматира, е по-голямо от 999, тогава може да се посочи в дефиницията на заместителя, че искаме да използваме запетая като разделител на хилядите:

```
source = "{:,d} символи"
print(source.format(5000))  # 5,000 символи
```

Освен това, заместителите могат да се използват и във f-низове:

```
n = 5000
source = f"{n:,d} символи"
print(source)  # 5,000 симбола
```



```
main.py
1 source = "{:,d} символи"
2 print(source.format(5000))  # 5,000 симболи
3 n = 5000
4
5 source = f"{n:,d} символи"
6 print(source)  # 5,000 симбола
```

Shell

```
5,000 симболи
5,000 симболи
>
```

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

За дробни числа, т.е. тези от тип `float`, преди кода за място след точката, може да се посочи колко знака в дробната част искаме да покажем:

```
number = 23.8589578
print("{:.2f}".format(number)) # 23.86
print("{:.3f}".format(number)) # 23.859
print("{:.4f}".format(number)) # 23.8590
print("{:,.2f}".format(10001.23554)) # 10,001.24
```

main.py		Run	Shell
1	<code>number = 23.8589578</code>		23.86
2	<code>print("{:.2f}".format(number))</code>	# 23.86	23.859
3	<code>print("{:.3f}".format(number))</code>	# 23.859	23.8590
4	<code>print("{:.4f}".format(number))</code>	# 23.8590	10,001.24
5	<code>print("{:,.2f}".format(10001.23554))</code>	# 10,001.24	>

Друг параметър позволява да се зададе минималната ширина на форматираната стойност в знаци:

```
print("{:10.2f}".format(23.8589578)) # 23.86
print("{:8d}".format(25)) # 25
```

main.py		Run	Shell
1	<code>print("{:10.2f}".format(23.8589578))</code>	# 23.86	23.86
2	<code>print("{:8d}".format(25))</code>	# 25	25

Подобен пример с f-низове:

```
n1 = 23.8589578
print(f"{n1:10.2f}") # 23.86
n2 = 25
print(f"{n2:8d}") # 25
```

main.py		Run	Shell
1	<code>n1 = 23.8589578</code>		23.86
2	<code>print(f"{n1:10.2f}")</code>	# 23.86	25
3	<code>n2 = 25</code>		>
4	<code>print(f"{n2:8d}")</code>	# 25	

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

За теглене на лихва е по-добре да се използва кода "%":

```
number = .12345
print("{:%}".format(number))      # 12.345000%
print("{:.0%}".format(number))    # 12%
print("{:.1%}".format(number))    # 12.3%
```

```
print(f"{number:%}")              # 12.345000%
print(f"{number:.0%}")            # 12%
print(f"{number:.1%}")            # 12.3%
```

main.py	Run	Shell
1 number = .12345		12.345000%
2 print("{:%}".format(number))	# 12.345000%	12%
3 print("{:.0%}".format(number))	# 12%	12.3%
4 print("{:.1%}".format(number))	# 12.3%	12.345000%
5		12%
6 print(f"{number:%}")	# 12.345000%	12.3%
7 print(f"{number:.0%}")	# 12%	>
8 print(f"{number:.1%}")	# 12.3%	

За показване на число в експоненциална нотация се използва заместителят "e":

```
number = 12345.6789
print("{:e}".format(number))      # 1.234568e+04
print("{:.0e}".format(number))    # 1e+04
print("{:.1e}".format(number))    # 1.2e+04
```

```
print(f"{number:e}")              # 1.234568e+04
print(f"{number:.0e}")            # 1e+04
print(f"{number:.1e}")            # 1.2e+04
```

main.py	Run	Shell
1 number = 12345.6789		1.234568e+04
2 print("{:e}".format(number))	# 1.234568e+04	1e+04
3 print("{:.0e}".format(number))	# 1e+04	1.2e+04
4 print("{:.1e}".format(number))	# 1.2e+04	1.234568e+04
5		1e+04
6 print(f"{number:e}")	# 1.234568e+04	1.2e+04
7 print(f"{number:.0e}")	# 1e+04	>
8 print(f"{number:.1e}")	# 1.2e+04	

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

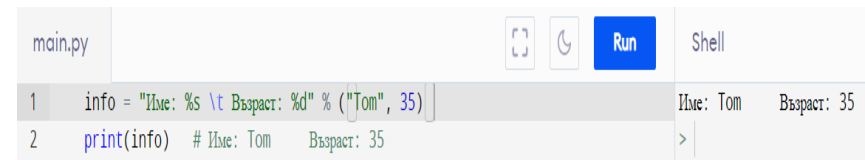
Форматиране без метода за форматиране

Има и друг начин за форматиране със следния синтаксис:

линия%(параметър1, параметър2,..параметърN)

Т.е., в началото има ред, който съдържа същите заместители, които бяха обсъдени по-горе (с изключение на заместващия %), след реда се поставя знак за процент % и след това списък със стойности, които се вмъкват в линията. Всъщност знакът за процента представлява операция, която води до нов ред:

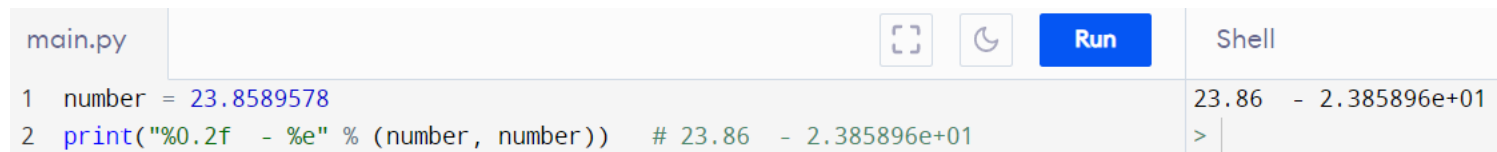
```
info = "Име: %s \t Възраст: %d" % ("Tom", 35)
print(info) # Име: Tom    Възраст: 35
```



```
main.py  Run  Shell
1 info = "Име: %s \t Възраст: %d" % ("Tom", 35)
2 print(info) # Име: Tom    Възраст: 35
Име: Tom    Възраст: 35
```

Заместителят е последван от знак за процент и за разлика от функцията за форматиране тук не се изискват фигурни скоби. Освен това тук се използват и методи за форматиране на числа:

```
number = 23.8589578
print("%0.2f - %e" % (number, number)) # 23.86 - 2.385896e+01
```



```
main.py  Run  Shell
1 number = 23.8589578
2 print("%0.2f - %e" % (number, number)) # 23.86 - 2.385896e+01
23.86 - 2.385896e+01
```

Програма за броене на думи

Нека да разгледаме работата с низове с помощта на малък пример, който представлява програма за броене на думи. Нека целият програмен код изглежда така:

```
# Програма за броене на думи във файл
```

```
import os
```

```
def get_words(filename):
```

```
    with open(filename, encoding="utf8") as file:
```

```
        text = file.read()
```

```
    text = text.replace("\n", " ")
```

```
    text = text.replace(",", "").replace(".", "").replace("?", "").replace("!", "")
```

```
    text = text.lower()
```

```
    words = text.split()
```

```
    words.sort()
```

```
    return words
```

Тема 8. Низ. Работа с низ. Основни методи. Форматиране

```
def get_words_dict(words):
    words_dict = dict()

    for word in words:
        if word in words_dict:
            words_dict[word] = words_dict[word] + 1
        else:
            words_dict[word] = 1
    return words_dict

def main():
    filename = input("Въведете пътя до файла: ")
    if not os.path.exists(filename):
        print("Посоченият файл не съществува")
    else:
        words = get_words(filename)
        words_dict = get_words_dict(words)
        print(f"Брой думи: {len(words)}")
        print(f"Брой уникални думи: {len(words_dict)}")
        print("Всички използвани думи:")
        for word in words_dict:
            print(word.ljust(20), words_dict[word])

if __name__ == "__main__":
    main()
```


Тема 8. Низ. Работа с низ. Основни методи. Форматиране

Тук във функцията `get_words()` се извършва първоначалното сегментиране на текста в думи. В този случай всички препинателни знаци се премахват, а преводите на дренажа се заменят с интервали. След това текстът се разделя на думи. Разделителят по подразбиране е интервал. По-нататък във функцията `get_words_dict()` се получава речник на думите, където ключът е уникална дума, а стойността е броят на срещанията на тази дума в текста. В основната функция се въвежда пътя до файла и се извикват по-горе дефинираните функции, както и изходът на цялата статистика. Конзолен изход на програмата:

Въведете пътя до файла: C:\SomeDir\hello.txt – Това е примерна програма с примерни данни!!!

Брой думи: 66

Брой уникални думи: 54

Всички използвани думи:

благодетел 2

в 1

общо 1

ти 1

горчица 1

нейната 1

ако 3

още 1

Използвана литература:

- [1]. Joakim Sundnes, Introduction to Scientific Programming with Python, Simula Springer Briefs on Computing, 2020
- [2]. Brian Heinold, A Practical Introduction to Python Programming, Department of Mathematics and Computer Science Mount St. Mary's University, 2012
- [3]. David Mertz, Functional Programming in Python, O'Reilly Media, 2015
- [4]. Steven Lott, Functional Python Programming, Published by Packt Publishing Ltd., 2015
- [5]. Mark Lutz, Learning Python, Fourth Edition, O'Reilly Media, 2009